BUSINESS ANALYSIS DOCUMENT

LOGISTICSPRO

OPERATIONS AND FINANCIAL MANAGEMENT SYSTEM

Contents

BUSINESS ANALYSIS DOCUMENT 1
LOGISTICSPRO 1
OPERATIONS AND FINANCIAL MANAGEMENT SYSTEM 1
1.1 Purpose of the Document 15
1.2 Business Context 15
1.3 Purpose of the System 16
1.4 Scope of the Document 17
1.5 Objectives of the System 17
1.6 Intended Audience 18
1.7 Product Perspective 19
1.8 Business Need 19
1.9 Assumptions 20
1.10 Constraints 20
1.11 Document Structure 21
1.12 Definitions, Acronyms, and Abbreviations 21
2.1 Overview 22
2.2 Business Requirement Categories 22
2.3 Detailed Business Requirements 23
BR-01: Authentication and Access Control 23
BR-02: Customer Management 23
BR-03: Vendor Management 24
BR-04: Event Management 24
BR-05: Job Card Management 24
BR-06: Operational Service Management 25
BR-07: Cost Management 25
BR-08: Batch Processing 26
BR-09: Payment Voucher Management 26
BR-10: Invoice Management 26

BR-11: Proforma Invoice Management 27
BR-12: Receipt Management 27
BR-13: Reporting and Analytics 27
2.4 Business Requirement Dependencies 28
2.5 Business Requirement Constraints 28
2.6 Business Requirement Acceptance Criteria 28
3.1 Overview 29
3.2 Functional Requirement Structure 29
3.3 Authentication and Authorization 29
FR-01: User Authentication 29
FR-02: Token Validation 30
3.4 Customer Management 31
FR-03: Create Customer 31
FR-04: Update Customer 31
FR-05: Retrieve Customer 32
3.5 Vendor Management 32
FR-06: Create Vendor 32
FR-07: Retrieve Vendor 33
3.6 Event Management 33
FR-08: Create Event 33
3.7 Job Card Management 34
FR-09: Create Job Card 34
FR-10: Update Job Card 34
FR-11: Close Job Card 34
3.8 Operational Service Management 35
FR-12: Add Service Item 35
FR-13: Update Service Item 35
FR-14: Remove Service Item 35
3.9 Cost Management 36

FR-15: Calculate Job Cost 36
3.10 Batch Processing 36
FR-16: Create Batch 36
3.11 Payment Voucher Management 36
FR-17: Create Payment Voucher 36
3.12 Invoice Management 37
FR-18: Create Invoice 37
FR-19: Void Invoice 37
3.13 Receipt Management 37
FR-20: Create Receipt 37
3.14 Reporting 37
FR-21: Generate Reports 37
4.1 Overview 38
4.2 Rule Classification 38
4.3 General System Rules 39
BRULE-01: Mandatory Data Integrity 39
BRULE-02: Valid Entity Reference 39
BRULE-03: Controlled Modification 39
BRULE-04: Audit Preservation 39
4.4 Authentication and Access Rules 40
BRULE-05: Authenticated Access Required 40
BRULE-06: Token Validity 40
BRULE-07: Identity Context 40
4.5 Master Data Rules 40
BRULE-08: Customer Record Completeness 40
BRULE-09: Vendor Record Completeness 41
BRULE-10: Event Date Consistency 41
BRULE-11: Protected Deletion of Master Data 41
BRULE-12: Active Usage Constraint 41

4.6 Job Card Lifecycle Rules 41
BRULE-13: Customer Requirement for Job Creation 41
BRULE-14: Job Card Identity 42
BRULE-15: Job Card Lifecycle Control 42
BRULE-17: Closed Job Restriction 42
BRULE-18: Job Closure Preconditions 42
BRULE-19: Deletion Restriction for Financially Used Jobs 43
4.7 Operational Service Rules 43
BRULE-20: Service-to-Job Relationship 43
BRULE-21: Service Type Validity 43
BRULE-22: Service Editability 43
BRULE-23: Service Deletion Restriction 44
BRULE-24: Mandatory Cost Value 44
BRULE-25: Transportation Route Validity 44
BRULE-26: Hotel Stay Validity 44
BRULE-27: Visa Service Validity 44
BRULE-28: Miscellaneous Service Clarity 45
4.8 Cost and Calculation Rules 45
BRULE-29: Non-Negative Cost Rule 45
BRULE-30: Recalculation on Change 45
BRULE-31: Aggregated Total Accuracy 45
BRULE-32: Financial Consistency 45
BRULE-33: Invalid Numeric Rejection 46
4.9 Batch Processing Rules 46
BRULE-34: Eligibility for Batching 46
BRULE-35: Single Batch Membership 46
BRULE-36: Empty Batch Prevention 46
BRULE-37: Batch Integrity 46
BRULE-38: Processed Batch Restriction 46

4.10 Payment Voucher Rules 47
BRULE-39: Valid Batch Requirement for Voucher Creation 47
BRULE-40: Voucher Amount Accuracy 47
BRULE-41: Duplicate Voucher Prevention 47
BRULE-42: Vendor Consistency for Voucher 47
BRULE-43: Voucher Traceability 47
4.11 Invoice and Proforma Invoice Rules 47
BRULE-44: Invoice Source Validity 47
BRULE-45: Invoice Amount Integrity 47
BRULE-46: Finalized Invoice Restriction 47
BRULE-47: Voided Invoice Preservation 48
BRULE-48: Proforma as Non-Final Billing 48
BRULE-49: Proforma Conversion Control 48
BRULE-50: Duplicate Billing Prevention 48
4.12 Receipt Rules 48
BRULE-51: Valid Invoice Requirement for Receipt 48
BRULE-52: Receipt Amount Limit 48
BRULE-53: Voided or Invalid Target Restriction 48
BRULE-54: Receipt Traceability 49
BRULE-55: Receipt Void Preservation 49
4.13 Reporting and Audit Rules 49
BRULE-56: Reporting Accuracy 49
BRULE-57: Profit and Loss Consistency 49
BRULE-58: Historical Integrity 49
BRULE-59: Uninvoiced Identification 49
BRULE-60: Auditability of Financial Events 49
4.14 Validation Message Requirements 50
BRULE-61: Clear Validation Feedback 50
BRULE-62: Consistent Error Handling 50

4.15 Cross-Module Consistency Rules 50
BRULE-63: Downstream Dependency Protection 50
BRULE-64: Operational-to-Financial Continuity 50
BRULE-65: No Contradictory States 50
4.16 Acceptance of Business Rules 50
5.1 Overview 51
5.2 Process 1: Customer and Job Initialization 51
5.3 Process 2: Add Operational Services to Job 53
5.4 Process 3: Job Cost Finalization and Closure 54
5.5 Process 4: Batch Creation for Financial Processing 55
5.6 Process 5: Payment Voucher Creation 56
5.7 Process 6: Invoice Generation 57
5.8 Process 7: Receipt Processing 58
5.9 Process 8: Reporting and Analysis 59
6.1 Overview 59
6.2 Data Entity Classification 60
6.3 Entity Definitions 61
6.3.1 Customer Entity 61
6.3.2 Vendor Entity 61
6.3.3 Event Entity 62
6.3.4 Job Card Entity 62
6.3.5 Service Item Entity (Generic) 63
6.3.6 Batch Entity 64
6.3.7 Payment Voucher Entity 64
6.3.8 Invoice Entity 65
6.3.9 Proforma Invoice Entity 65
6.3.10 Receipt Entity 65
6.4 Entity Relationships 66
6.5 Data Integrity Rules 66

6.6 Data Lifecycle Considerations 66
7.1 Overview 67
7.2 Performance Requirements 68
7.2.1 General Performance Expectation 68
7.2.2 Response Time 68
7.2.3 Concurrent Usage 68
7.2.4 Efficient Data Access 69
7.2.5 Reporting Performance 69
7.3 Availability Requirements 69
7.3.1 Service Availability 69
7.3.2 Controlled Downtime 69
7.3.3 Recovery from Failure 69
7.3.4 Dependency Awareness 69
7.4 Reliability Requirements 69
7.4.1 Transaction Reliability 69
7.4.2 Fault Handling 70
7.4.3 Stable Financial Processing 70
7.4.4 Idempotency Consideration 70
7.4.5 Consistent Operation Under Load 70
7.5 Security Requirements 70
7.5.1 Authenticated Access 70
7.5.2 Authorization Enforcement 70
7.5.3 Secure Token Handling 70
7.5.4 Data Protection 71
7.5.5 Input Validation 71
7.5.6 Error Message Control 71
7.5.7 Audit-Sensitive Financial Security 71
7.5.8 Secure Configuration 71
7.6 Data Integrity Requirements 71

7.6.1 Referential Integrity 71
7.6.2 Transactional Consistency 71
7.6.3 Financial Integrity 72
7.6.4 Prevention of Orphan Records 72
7.6.5 Controlled State Changes 72
7.6.6 Historical Preservation 72
7.7 Scalability Requirements 72
7.7.1 Growth in Transaction Volume 72
7.7.2 Growth in Data Volume 72
7.7.3 Module Expansion 72
7.7.4 Reporting Scalability 72
7.8 Maintainability Requirements 73
7.8.1 Modular Design 73
7.8.2 Readable and Structured Code 73
7.8.3 Business Rule Maintainability 73
7.8.4 Configurability 73
7.8.5 Test Support 73
7.9 Auditability and Traceability Requirements 73
7.9.1 Financial Audit Support 73
7.9.2 Historical Visibility 73
7.9.3 User Action Traceability 74
7.9.4 Non-Destructive Financial Reversal 74
7.10 API Usability Requirements 74
7.10.1 Consistent Request and Response Structure 74
7.10.2 Clear Error Responses 74
7.10.3 Predictable Endpoint Behavior 74
7.10.4 Documentation Readiness 74
7.11 Compatibility and Interoperability Requirements 74
7.11.1 Standard API Communication 74

7.11.2 Data Exchange Compatibility 75
7.11.3 Environment Portability 75
7.12 Logging and Monitoring Requirements 75
7.12.1 Operational Logging 75
7.12.2 Error Monitoring 75
7.12.3 Business Event Visibility 75
7.12.4 Sensitive Data Protection in Logs 75
7.13 Backup and Recovery Requirements 75
7.13.1 Data Backup 75
7.13.2 Recovery Capability 76
7.13.3 Recovery Integrity 76
7.14 Compliance and Governance Considerations 76
7.14.1 Controlled Financial Behavior 76
7.14.2 Data Handling Responsibility 76
7.14.3 Change Control 76
7.15 Non-Functional Acceptance Criteria 76
8.1 Overview 77
8.2 API Design Principles 77
8.3 Interface Consumers 78
8.3.1 Internal Business Applications 78
8.3.2 Administrative Tools 78
8.3.3 Reporting or Integration Services 78
8.3.4 Future External Consumers 78
8.4 Communication Protocol Requirements 79
8.5 HTTP Method Usage 79
8.6 Resource Structure Requirements 80
8.7 Request Requirements 81
8.7.1 Request Structure 81
8.7.2 Identifier Usage 81

8.7.3 Filtering and Query Parameters 82
8.8 Status Code Requirements 82
8.9 Authentication Interface Requirements 83
8.9.1 Authentication Endpoint Behavior 83
8.9.2 Authenticated Request Behavior 83
8.10 Authorization Interface Requirements 83
8.11 Validation and Error Response Requirements 84
8.11.1 Input Validation 84
8.11.2 Error Response Structure 84
8.11.3 Business Rule Violation Handling 84
8.12 Pagination, Sorting, and Search Requirements 85
8.13 Date, Time, and Numeric Interface Requirements 85
8.14 Financial Endpoint Requirements 86
8.15 Reporting Interface Requirements 87
8.16 Versioning Requirements 87
8.17 Documentation Requirements 87
8.18 Interface Logging and Traceability Requirements 88
8.19 API Acceptance Criteria 88
9.1 Overview 89
9.2 Assumptions 89
9.2.1 Business User Assumptions 89
9.2.2 Data Assumptions 90
9.2.3 Technical Assumptions 90
9.2.4 Operational Assumptions 90
9.3 Dependencies 91
9.3.1 Authentication Dependency 91
9.3.2 Database Dependency 91
9.3.3 Configuration Dependency 91
9.3.4 Infrastructure Dependency 91

9.3.5 Data Quality Dependency 92
9.3.6 Business Process Dependency 92
9.4 Risks 92
9.4.1 Data Integrity Risk 92
9.4.2 Financial Accuracy Risk 93
9.4.3 Unauthorized Access Risk 94
9.4.4 Duplicate Processing Risk 94
9.4.5 Concurrency Risk 95
9.4.6 Reporting Accuracy Risk 96
9.4.7 Availability and Dependency Failure Risk 96
9.4.8 Change Impact Risk 97
9.5 Exception Handling Requirements 97
9.5.1 General Exception Handling Principle 97
9.5.2 Validation Exceptions 98
9.5.2 Validation Exceptions 98
9.5.4 Not Found Exceptions 99
9.5.4 Not Found Exceptions 99
9.5.6 Duplicate Processing Exceptions 99
9.5.7 Dependency Failure Exceptions 100
9.5.8 Unexpected Internal Exceptions 100
9.6 Exception Handling Principles by Business Area 100
9.6.1 Master Data Exceptions 100
9.6.2 Operational Exceptions 101
9.6.3 Financial Exceptions 101
9.6.4 Reporting Exceptions 101
9.7 Risk Monitoring and Review 101
9.8 Acceptance Criteria for Assumptions, Dependencies, Risks, and Exception Handling ... 101
10.1 Overview 102
10.2 Traceability Structure 102

10.3 Traceability Matrix 103
10.4 Traceability Coverage Validation 103
10.5 Impact Analysis Usage 104
10.6 Testing Alignment 104
10.7 Maintenance and Change Control 104
10.8 Traceability Acceptance Criteria 105
11.1 Overview 105
11.2 System-Level Acceptance Criteria 106
11.3 Module-Level Acceptance Criteria 106
11.3.1 Customer and Vendor Management 106
11.3.2 Event Management 106
11.3.3 Job Card Management 107
11.3.4 Service Management 107
11.3.5 Batch Processing 107
11.3.6 Payment Voucher Management 107
11.3.7 Invoice Management 108
11.3.8 Receipt Management 108
11.3.9 Reporting 108
11.4 Business Flow Acceptance Criteria 108
11.5 Data Integrity Acceptance Criteria 109
11.6 Security Acceptance Criteria 109
11.7 API Behavior Acceptance Criteria 109
11.8 Performance and Stability Acceptance Criteria 110
11.9 Exception Handling Acceptance Criteria 110
11.10 Regression and Consistency Acceptance Criteria 110
11.11 Final Acceptance Condition 111
12.1 Overview 111
12.2 Glossary of Terms 111
12.3 Abbreviations 112

12.4 Entity Reference Summary 113
12.5 Status Definitions 114
12.6 Error Category Definitions.....115
12.7 Document Structure Summary.....115
12.8 Usage Guidelines.....116

1. Introduction

1.1 Purpose of the Document

This Business Analysis Document defines the business requirements, functional expectations, constraints, and operational rules for the LogisticsPro API. The purpose of this document is to provide a clear and structured specification of the system so that stakeholders, business analysts, architects, developers, testers, and support teams share a common understanding of what the API must deliver.

The document serves as a reference for the design and implementation of the LogisticsPro backend system. It describes the business context in which the API operates, the major business processes it supports, the entities it manages, and the rules that govern system behavior. It is intended to reduce ambiguity during development by translating business needs into precise and traceable requirements.

This document also supports communication across technical and non-technical stakeholders by presenting the system in business terms rather than only in source code or technical design language. It establishes the baseline against which the API can be implemented, validated, enhanced, and maintained over time.

1.2 Business Context

Organizations operating in logistics and service delivery environments often manage a combination of operational activities and financial processes that are tightly connected. These activities commonly include customer handling, event coordination, job execution, vendor-related services, cost tracking, invoice preparation, payment collection, and financial reconciliation. In such environments, a single business transaction may affect several parts of the system at once. For example, creating a job for a customer may later lead to transportation bookings, hotel arrangements, visa handling, vendor charges, invoice generation, receipt recording, and reporting on profitability.

Without a properly structured backend system, these workflows can become fragmented, inconsistent, and difficult to manage. Operational records may not align with financial records,

manual tracking may increase error rates, and business users may struggle to obtain accurate visibility into job progress, uninvoiced work, vendor obligations, and final profitability.

The LogisticsPro API is intended to address these challenges by providing a centralized backend platform that supports the creation, maintenance, and tracking of operational and financial data. The API enables consistent handling of customer-related jobs and associated services while preserving the relationships between business entities and the downstream financial outcomes generated from them. As a result, the system supports both day-to-day operational control and structured financial processing.

1.3 Purpose of the System

The LogisticsPro API is designed to function as the backend business layer for a logistics and service management platform. Its primary purpose is to manage the lifecycle of operational work and ensure that related financial activities are recorded and processed in a controlled and traceable manner.

At a business level, the system is intended to support the following goals:

- maintain core business records such as customers, vendors, and events
- create and manage job cards representing operational work
- attach operational service items such as transportation, hotel, visa, and miscellaneous services to jobs
- track service costs and maintain consistency of financial values
- group operational items into batches for downstream financial processing
- create and manage payment vouchers, invoices, proforma invoices, and receipts
- produce operational and financial reporting outputs required for business monitoring and decision-making

The API is not merely a data storage interface. It is expected to enforce business rules, validate process conditions, preserve entity relationships, and ensure that business transactions occur in the correct sequence.

1.4 Scope of the Document

This document covers the business and functional requirements for the LogisticsPro API. It includes the major modules, business entities, workflows, validations, business constraints, and user interactions required to support logistics and service-related operations together with financial processing.

The scope of this document includes:

- authentication and access control requirements for protected API access
- customer management requirements
- vendor management requirements
- event management requirements
- job card management requirements
- operational service management requirements
- batch processing requirements
- payment voucher management requirements
- invoice and proforma invoice management requirements
- receipt management requirements
- reporting and profit-and-loss related requirements
- cross-cutting business rules and validation requirements

This document focuses on backend API behavior and business logic. It defines what the system must do and the rules under which it must operate. It does not describe user interface designs, visual layouts, or frontend-specific behavior unless necessary to explain a business requirement.

1.5 Objectives of the System

The LogisticsPro API is expected to achieve several operational and business objectives.

First, it must provide a reliable way to manage the full lifecycle of jobs associated with customers and events. This includes the creation of job cards, the addition of service components, and the controlled progression of work through operational stages.

Second, it must maintain traceability between operational data and financial data. Costs generated through operational services must be accurately reflected in downstream financial documents such as vouchers and invoices. This ensures that business users can trust the integrity of billing and reporting outcomes.

Third, it must standardize the handling of logistics-related services by ensuring that transportation, hotel, visa, and miscellaneous activities are recorded in a consistent format and subject to clearly defined validation rules.

Fourth, it must support accurate financial processing by managing invoice creation, receipt recording, proforma handling, payment voucher generation, and related reporting functions.

Finally, it must provide a stable and governed backend foundation that can be extended and maintained as business needs evolve.

1.6 Intended Audience

This document is intended for multiple stakeholder groups involved in the planning, implementation, and operation of the LogisticsPro API.

Business stakeholders use this document to confirm that the system aligns with operational and financial needs.

Business analysts use it to refine, validate, and trace requirements across project stages.

Solution architects and technical leads use it to translate business requirements into technical architecture and service design.

Developers use it as a primary reference when implementing API endpoints, domain logic, validations, and entity relationships.

Quality assurance teams use it to understand the expected behavior of the system and verify that the delivered API meets the stated requirements.

Support and maintenance teams use it to understand the intended logic of the system when troubleshooting incidents, reviewing enhancements, or onboarding new team members.

1.7 Product Perspective

The LogisticsPro API is a backend application that acts as the central business processing layer for logistics and financial operations. It is expected to expose REST-based endpoints that allow authorized clients to create, retrieve, update, remove, and process business records across multiple modules.

The system is composed of interdependent business areas rather than isolated CRUD functions. For example, customer and event data provide context for job cards; job cards serve as containers for operational services; operational service records influence cost calculations; cost and service data feed batch processing and financial artifacts; financial transactions then affect reporting and profitability views.

Because of this interconnected nature, the API must be developed as a rules-driven business system rather than a simple collection of independent endpoints. Changes in one module may affect behavior in other modules, and the implementation must preserve consistency across the overall process chain.

1.8 Business Need

The organization requires a system that can manage operational and financial workflows in a more structured, centralized, and traceable manner. Existing manual or loosely connected processes can lead to duplicated effort, inconsistent records, delayed billing, incomplete reporting, and reduced visibility into actual profitability.

There is a need for a backend system that can:

- organize customer-related work in a consistent format
- link service activities to their financial implications
- prevent invalid or duplicate processing
- maintain accurate billing and payment records

- provide reliable reporting for operations and finance teams

The LogisticsPro API addresses this need by formalizing the core business workflows and applying controlled business logic to the management of jobs, services, and financial transactions.

1.9 Assumptions

The following assumptions apply to this document and to the expected implementation of the system:

- the system is primarily intended for authorized internal users or trusted client applications
- authentication is required for access to protected business functions
- customers, vendors, and events must exist before they can be referenced by downstream transactions
- financial transactions must remain traceable for audit and business review purposes
- the backend is responsible for enforcing business rules and validation logic
- reporting outputs depend on the accuracy and completeness of operational and financial records entered into the system

These assumptions should be reviewed during detailed design and implementation to ensure alignment with actual deployment and business practices.

1.10 Constraints

The development and operation of the LogisticsPro API are subject to several business and system constraints.

The system must preserve the relationship between operational records and financial records. Business actions must not result in orphaned, duplicated, or inconsistent data.

The system must restrict invalid transitions or updates. For example, records that have already been financially processed may require stricter control over subsequent modification or deletion.

The system must support auditability of important financial actions such as invoice voiding, receipt recording, and payment voucher generation.

The system is focused on backend operations and does not, within this document, include requirements for customer-facing portals, public booking features, or external payment gateway integrations unless explicitly defined elsewhere.

1.11 Document Structure

This document is organized to provide a complete business and functional understanding of the LogisticsPro API.

The introduction establishes the purpose, business context, scope, and objectives of the system. Subsequent sections define the business domain, stakeholders, functional modules, business processes, entity requirements, business rules, validations, non-functional expectations, and acceptance criteria. Together, these sections provide a structured foundation for implementation and lifecycle management of the API.

1.12 Definitions, Acronyms, and Abbreviations

Term	**Definition**
API	Application Programming Interface
Customer	A business party receiving services from the organization
Vendor	A supplier or service provider used for operational fulfillment
Event	A business grouping or context under which jobs may be organized
Job Card	The primary operational record representing a unit of work
Service Item	A service component linked to a job, such as transport, hotel, visa, or miscellaneous
Batch	A collection of eligible items grouped for financial processing

Payment Voucher	A financial record used to process or record vendor payments
Invoice	A billing document issued to a customer
Proforma Invoice	A preliminary billing document issued before final invoicing
Receipt	A record of payment received against a financial obligation
P&L	Profit and Loss
JWT	JSON Web Token used for authentication and authorization

2. Business Requirements

2.1 Overview

This section defines the high-level business requirements that the LogisticsPro API must satisfy. These requirements describe the expected business capabilities of the system and the outcomes it must support. Each requirement is expressed in a manner that allows traceability to functional specifications, implementation, and validation activities.

The business requirements are grouped based on major business domains within the system, including authentication, master data management, operational processing, financial processing, and reporting.

2.2 Business Requirement Categories

The system must support the following key business capability areas:

- Secure system access and user authentication
- Management of core business entities such as customers, vendors, and events
- Creation and lifecycle management of job cards
- Recording and maintenance of operational service items

- Cost tracking and aggregation
- Grouping of operational items for financial processing
- Processing of vendor payments through payment vouchers
- Customer billing through invoices and proforma invoices
- Recording of payments through receipts
- Generation of operational and financial reports

2.3 Detailed Business Requirements

BR-01: Authentication and Access Control

The system must ensure that only authorized users can access protected business functionalities.

Requirements:

- The system must require authentication before granting access to business operations
- The system must validate user identity for each request
- The system must restrict unauthorized access to all protected endpoints

Expected Outcome:

All business operations are performed only by authenticated and authorized users.

BR-02: Customer Management

The system must provide the ability to manage customer information used in operational and financial processes.

Requirements:

- The system must allow creation of new customer records
- The system must allow updating of existing customer details
- The system must allow retrieval of customer information
- The system must maintain uniqueness and integrity of customer data

Expected Outcome:

Accurate and consistent customer records are available for all downstream processes.

BR-03: Vendor Management

The system must support the management of vendors who provide operational services.

Requirements:

- The system must allow creation and maintenance of vendor records
- The system must associate vendors with service types
- The system must allow retrieval of vendor information

Expected Outcome:

Vendors are properly maintained and available for assignment in operational services.

BR-04: Event Management

The system must allow the creation and management of events that group operational activities.

Requirements:

- The system must allow creation of events
- The system must allow updating event details
- The system must allow retrieval of event information

Expected Outcome:

Events can be used to logically group and organize job cards.

BR-05: Job Card Management

The system must support the creation and lifecycle management of job cards representing operational work.

Requirements:

- The system must allow creation of job cards linked to customers

- The system must allow updating of job card details
- The system must allow retrieval of job cards
- The system must allow deletion of job cards under valid conditions
- The system must support opening and closing of job cards

Expected Outcome:

Operational work is tracked through structured job cards with controlled lifecycle states.

BR-06: Operational Service Management

The system must support recording and management of service items associated with job cards.

Requirements:

- The system must allow adding service items to a job card
- The system must allow updating service items
- The system must allow removal of service items
- The system must support multiple service types including transportation, hotel, visa, and miscellaneous

Expected Outcome:

All operational activities are captured as structured service records linked to job cards.

BR-07: Cost Management

The system must ensure accurate tracking and aggregation of costs associated with service items.

Requirements:

- The system must maintain cost values for each service item
- The system must calculate total cost at job level
- The system must update cost calculations dynamically based on changes

Expected Outcome:

Financial values accurately reflect operational activities.

BR-08: Batch Processing

The system must group operational service items into batches for financial processing.

Requirements:

- The system must allow selection of eligible items for batching
- The system must allow creation and management of batches
- The system must prevent duplication of items across batches

Expected Outcome:

Operational items are efficiently grouped for downstream financial workflows.

BR-09: Payment Voucher Management

The system must support the creation and management of payment vouchers for vendor payments.

Requirements:

- The system must allow creation of payment vouchers linked to batch items
- The system must allow retrieval of payment voucher information
- The system must ensure correctness of payment amounts

Expected Outcome:

Vendor payment obligations are recorded and managed accurately.

BR-10: Invoice Management

The system must support billing of customers through invoices.

Requirements:

- The system must allow creation of invoices
- The system must allow retrieval of invoices

- The system must allow voiding of invoices under valid conditions

Expected Outcome:

Customer billing is handled in a structured and controlled manner.

BR-11: Proforma Invoice Management

The system must support preliminary billing through proforma invoices.

Requirements:

- The system must allow creation of proforma invoices
- The system must allow conversion of proforma invoices into final invoices

Expected Outcome:

Preliminary billing can be performed prior to final financial processing.

BR-12: Receipt Management

The system must support recording of payments received from customers.

Requirements:

- The system must allow creation of receipts
- The system must allow linking of receipts to invoices
- The system must support voiding of receipts

Expected Outcome:

Payments are accurately recorded and linked to corresponding financial documents.

BR-13: Reporting and Analytics

The system must provide reporting capabilities for operational and financial data.

Requirements:

- The system must generate operational reports
- The system must generate historical reports

- The system must generate uninvoiced reports
- The system must generate profit and loss reports

Expected Outcome:

Business users have access to accurate and meaningful insights.

2.4 Business Requirement Dependencies

- Job Card Management depends on Customer and Event Management
- Service Management depends on Job Card Management
- Cost Management depends on Service Management
- Batch Processing depends on Service Management
- Payment Voucher Management depends on Batch Processing
- Invoice Management depends on Job Card and Service data
- Receipt Management depends on Invoice Management
- Reporting depends on all modules

2.5 Business Requirement Constraints

- Business operations must follow defined process sequences
- Financial transactions must be traceable and auditable
- Invalid operations must be rejected by the system
- Data consistency must be maintained across all modules

2.6 Business Requirement Acceptance Criteria

Each business requirement is considered satisfied when:

- The functionality is implemented according to the defined requirement
- All relevant business rules are enforced

- The system prevents invalid operations
- Outputs match expected business outcomes
- Data consistency is maintained across modules

3. Functional Requirements

3.1 Overview

This section defines the detailed functional requirements of the LogisticsPro API. These requirements specify the system behavior in terms of inputs, processing logic, and expected outputs. Each requirement describes what the system must do under specific conditions and is structured to support implementation, validation, and traceability.

Functional requirements are grouped based on system modules and are aligned with the business requirements defined in the previous section.

3.2 Functional Requirement Structure

Each functional requirement is defined using the following structure:

- **ID**: Unique identifier
- **Description**: What the system must do
- **Inputs**: Required data
- **Processing**: System behavior
- **Outputs**: Expected result
- **Validation Rules**: Constraints and checks

3.3 Authentication and Authorization

FR-01: User Authentication

Description

The system must authenticate users before allowing access to protected resources.

Inputs

- Username / Email
- Password

Processing

- Validate credentials against stored records
- Generate authentication token

Outputs

- Success: Authentication token returned
- Failure: Unauthorized response

Validation Rules

- Credentials must be valid
- Invalid login attempts must be rejected

FR-02: Token Validation

Description

The system must validate authentication tokens for all protected API requests.

Inputs

- JWT token

Processing

- Verify token signature
- Check token expiration
- Extract user identity

Outputs

- Success: Request proceeds

- Failure: Unauthorized response

Validation Rules

- Token must not be expired
- Token must be valid and untampered

3.4 Customer Management

FR-03: Create Customer

Description

The system must allow creation of a new customer record.

Inputs

- Customer Name
- Contact Details

Processing

- Validate input data
- Store customer record

Outputs

- Success: Customer created with ID
- Failure: Validation error

Validation Rules

- Name is mandatory
- Duplicate entries must be prevented (if applicable)

FR-04: Update Customer

Description

The system must allow updating existing customer details.

Inputs

- Customer ID
- Updated fields

Processing

- Verify customer exists
- Update record

Outputs

- Success: Updated customer
- Failure: Not found / validation error

FR-05: Retrieve Customer

Description

The system must allow retrieval of customer details.

Inputs

- Customer ID

Processing

- Fetch customer record

Outputs

- Success: Customer details
- Failure: Not found

3.5 Vendor Management

FR-06: Create Vendor

Description

The system must allow creation of vendor records.

Inputs

- Vendor Name
- Service Type

Processing

- Validate and store vendor

Outputs

- Success: Vendor created

FR-07: Retrieve Vendor

Description

The system must allow retrieval of vendor details.

3.6 Event Management

FR-08: Create Event

Description

The system must allow creation of event records.

Inputs

- Event Name
- Start Date
- End Date

Validation Rules

- End Date must be greater than or equal to Start Date

3.7 Job Card Management

FR-09: Create Job Card

Description

The system must allow creation of a job card.

Inputs

- Customer ID
- Event ID (optional)

Processing

- Validate customer exists
- Create job record

Outputs

- Job ID generated

FR-10: Update Job Card

Description

The system must allow updating job card details.

Validation Rules

- Job must not be closed

FR-11: Close Job Card

Description

The system must allow closing of job cards.

Processing

- Update status to Closed

Validation Rules

- Closed jobs cannot be modified

3.8 Operational Service Management

FR-12: Add Service Item

Description

The system must allow adding service items to a job.

Inputs

- Job ID
- Service Type
- Cost

Processing

- Validate job exists
- Add service record

Validation Rules

- Cost must be ≥ 0

FR-13: Update Service Item

Description

The system must allow updating service details.

FR-14: Remove Service Item

Description

The system must allow removal of service items.

Validation Rules

- Cannot remove if already invoiced

3.9 Cost Management

FR-15: Calculate Job Cost

Description

The system must calculate total cost of a job.

Processing

- Sum all service costs

Output

- Total cost

3.10 Batch Processing

FR-16: Create Batch

Description

The system must group service items into a batch.

Validation Rules

- Items must not be duplicated

3.11 Payment Voucher Management

FR-17: Create Payment Voucher

Description

The system must create payment vouchers for vendors.

Inputs

- Batch ID

Processing

- Calculate payable amount

3.12 Invoice Management

FR-18: Create Invoice

Description

The system must generate invoices.

Validation Rules

- Must link to valid job

FR-19: Void Invoice

Description

The system must allow voiding invoices.

3.13 Receipt Management

FR-20: Create Receipt

Description

The system must record payments.

Validation Rules

- Amount must not exceed invoice

3.14 Reporting

FR-21: Generate Reports

Description

The system must generate reports.

Types

- Operational
- Financial
- P&L

4. Business Rules and Validation Requirements

4.1 Overview

This section defines the business rules, validation rules, and operational constraints that govern the behavior of the LogisticsPro API. These rules ensure that the system processes data consistently, preserves business integrity, and prevents invalid or contradictory operations.

Business rules are mandatory conditions derived from the business domain. They apply across modules such as master data management, job processing, operational services, financial processing, and reporting. Validation requirements define the checks that must be applied to incoming data before business transactions are accepted.

The purpose of this section is to remove ambiguity from the functional requirements by stating exactly what conditions must hold true before a process can be executed, what transitions are allowed, and what actions must be restricted.

4.2 Rule Classification

The business rules in this document are grouped into the following categories:

- General system rules
- Authentication and access rules
- Master data rules
- Job card lifecycle rules
- Operational service rules
- Cost and calculation rules
- Batch processing rules
- Payment voucher rules
- Invoice and proforma invoice rules
- Receipt rules

- Reporting and audit rules

Each rule is assigned an identifier for traceability.

4.3 General System Rules

BRULE-01: Mandatory Data Integrity

All records created, updated, or processed by the system must preserve referential and transactional integrity across related business entities.

Explanation:
The system must not allow orphaned records, broken references, or inconsistent cross-module updates.

BRULE-02: Valid Entity Reference

A transaction must not reference a customer, vendor, event, job card, batch, invoice, or other entity unless that entity already exists and is valid for use.

Explanation:
The system must verify the existence of referenced entities before processing the request.

BRULE-03: Controlled Modification

Records that have reached a finalized, closed, processed, or voided state must be subject to controlled modification rules.

Explanation:
Not every record can remain editable forever. Some records must become restricted after reaching a defined state.

BRULE-04: Audit Preservation

Where financial or operational actions have audit significance, the system must preserve transaction history and avoid destructive processing that removes historical traceability.

Explanation:
Important actions such as voiding financial records should preserve evidence of prior existence and status.

4.4 Authentication and Access Rules

BRULE-05: Authenticated Access Required

All protected API operations must require successful authentication.

Validation Requirement

- Requests without valid credentials or token must be rejected.

BRULE-06: Token Validity

The system must reject expired, malformed, tampered, or otherwise invalid authentication tokens.

Validation Requirement

- Token structure, expiration, and signature must be validated before request processing.

BRULE-07: Identity Context

Every authorized transaction must be processed within the context of the authenticated user identity.

Explanation:
Where required, the system must associate changes or actions with the authenticated user.

4.5 Master Data Rules

BRULE-08: Customer Record Completeness

A customer record must contain all mandatory attributes required to identify and manage the customer in operational and financial processes.

Validation Requirements

- Customer name must be provided.
- Required business contact fields must not be blank if marked mandatory by the domain model.

BRULE-09: Vendor Record Completeness

A vendor record must include the minimum information required to identify the vendor and associate the vendor with supported services.

Validation Requirements

- Vendor name must be provided.
- Vendor must be classifiable for operational usage.

BRULE-10: Event Date Consistency

Where an event includes start and end dates, the end date must not occur before the start date.

Validation Requirements

- EndDate >= StartDate

BRULE-11: Protected Deletion of Master Data

A customer, vendor, or event must not be deleted if it is actively referenced by downstream business transactions unless the system explicitly supports safe deactivation instead of deletion.

Explanation:
This prevents broken references and historical inconsistencies.

BRULE-12: Active Usage Constraint

Only active or valid master data records should be available for new downstream transactions.

Explanation:
Inactive entities may remain visible historically but should not necessarily be eligible for new business use.

4.6 Job Card Lifecycle Rules

BRULE-13: Customer Requirement for Job Creation

A job card must be linked to a valid customer.

Validation Requirements

- Job creation must fail if the customer reference is missing or invalid.

BRULE-14: Job Card Identity

Each job card must have a unique identifier.

Explanation:
The system must ensure that each job card is uniquely traceable.

BRULE-15: Job Card Lifecycle Control

A job card must operate within defined lifecycle states.

Minimum Expected States

- Open
- Closed

BRULE-17: Closed Job Restriction

A closed job card must not accept new operational service items or unrestricted modifications.

Explanation:
Closing a job card indicates operational completion or administrative lock.

BRULE-18: Job Closure Preconditions

The system may require certain business conditions to be satisfied before a job card can be closed.

Possible Conditions

- Required operational data completed
- Financial dependencies reviewed
- No invalid incomplete service lines

Where such preconditions are enforced by the implementation, the API must validate them prior to closure.

BRULE-19: Deletion Restriction for Financially Used Jobs

A job card must not be deleted if it has already contributed to invoices, batches, vouchers, receipts, or other financially significant downstream records.

Explanation:
Historical financial traceability must be preserved.

4.7 Operational Service Rules

Operational services include transportation, hotel, visa, and miscellaneous service entries linked to a job card.

BRULE-20: Service-to-Job Relationship

Every operational service item must belong to a valid job card.

Validation Requirements

- Service creation must fail if JobId is invalid or missing.

BRULE-21: Service Type Validity

A service item must be classified under a valid supported service category.

Supported Types

- Transportation
- Hotel
- Visa
- Miscellaneous

BRULE-22: Service Editability

A service item may only be edited if its parent job card remains eligible for modification and the service has not been locked by downstream financial processing.

BRULE-23: Service Deletion Restriction

A service item must not be removed if it has already been included in a finalized financial process unless the system supports controlled reversal procedures.

BRULE-24: Mandatory Cost Value

Every service item that contributes to business cost must contain a valid cost value.

Validation Requirements

- Cost must be numeric.
- Cost must not be negative.

BRULE-25: Transportation Route Validity

For transportation entries, pickup and drop-off information must be valid and meaningful.

Validation Requirements

- Pickup and drop-off must not both be blank.
- Where both are provided, they should not represent the same unusable route unless explicitly allowed.

BRULE-26: Hotel Stay Validity

For hotel entries, stay duration must be logically valid.

Validation Requirements

- Check-out date must be later than or equal to check-in date, depending on the defined booking model.
- Room-related quantities must be positive if quantity fields are used.

BRULE-27: Visa Service Validity

Visa-related services must include the required identifying or processing information defined by the service model.

Explanation:
Mandatory visa-specific fields must be validated before save.

BRULE-28: Miscellaneous Service Clarity

Miscellaneous items must contain sufficient descriptive information to justify their existence and financial value.

Validation Requirements

- Description must not be empty.
- Cost must remain valid.

4.8 Cost and Calculation Rules

BRULE-29: Non-Negative Cost Rule

All cost-bearing service entries must have non-negative amounts.

BRULE-30: Recalculation on Change

The total cost of a job card must be recalculated whenever a cost-bearing service item is added, updated, or removed.

BRULE-31: Aggregated Total Accuracy

The total job cost must equal the sum of all valid included service item costs associated with that job.

BRULE-32: Financial Consistency

Operational totals used in downstream financial processing must remain consistent with the relevant included service records.

Explanation:
The system must avoid mismatches between operational values and financial values derived from them.

BRULE-33: Invalid Numeric Rejection

The system must reject invalid numeric values such as non-numeric costs, malformed decimals, or values outside supported limits.

4.9 Batch Processing Rules

BRULE-34: Eligibility for Batching

Only eligible operational items may be added to a batch.

Typical Eligibility Conditions

- Item exists
- Item is valid
- Item is not already processed in another conflicting financial workflow

BRULE-35: Single Batch Membership

An item must not belong to multiple active or conflicting batches when the business process expects exclusive grouping.

BRULE-36: Empty Batch Prevention

The system must not create or persist a financial batch without at least one valid item.

BRULE-37: Batch Integrity

All items included in a batch must remain valid and traceable to their source records.

BRULE-38: Processed Batch Restriction

Once a batch has been processed into downstream financial transactions, unrestricted editing or item replacement should not be allowed.

4.10 Payment Voucher Rules

BRULE-39: Valid Batch Requirement for Voucher Creation

A payment voucher must be linked to valid batch data or another valid source of payable operational items.

BRULE-40: Voucher Amount Accuracy

The value of a payment voucher must match the payable amount derived from the included operational items according to business rules.

BRULE-41: Duplicate Voucher Prevention

The system must prevent duplicate or conflicting payment voucher creation for the same payable business content unless intentional partial or revised voucher behavior is explicitly supported.

BRULE-42: Vendor Consistency for Voucher

Where the voucher is vendor-specific, the included payable items must align with the vendor associated with the voucher.

BRULE-43: Voucher Traceability

A voucher must remain traceable to the items, batches, or services that generated the payable amount.

4.11 Invoice and Proforma Invoice Rules

BRULE-44: Invoice Source Validity

An invoice must be created only from valid billable business content.

BRULE-45: Invoice Amount Integrity

Invoice totals must accurately reflect the chargeable content from the underlying operational and financial records.

BRULE-46: Finalized Invoice Restriction

Once finalized, an invoice must not be freely modified.

Explanation:
Subsequent correction should occur through controlled actions such as voiding or issuance of replacement records if supported.

BRULE-47: Voided Invoice Preservation

Voiding an invoice must preserve the invoice record and its historical relationship rather than erase it.

BRULE-48: Proforma as Non-Final Billing

A proforma invoice must be treated as a preliminary billing document and not as a finalized financial settlement record.

BRULE-49: Proforma Conversion Control

Where conversion from proforma invoice to final invoice is supported, the conversion must preserve business consistency and avoid duplicated billing.

BRULE-50: Duplicate Billing Prevention

The system must prevent double invoicing of the same billable content unless explicitly allowed by the business model.

4.12 Receipt Rules

BRULE-51: Valid Invoice Requirement for Receipt

A receipt must be linked to a valid payable invoice or supported financial source.

BRULE-52: Receipt Amount Limit

The cumulative total of receipts applied to an invoice must not exceed the allowed receivable amount for that invoice unless overpayment handling is explicitly supported.

BRULE-53: Voided or Invalid Target Restriction

The system must prevent receipt application to a voided, invalid, or otherwise non-receivable financial document unless business rules explicitly permit it.

BRULE-54: Receipt Traceability

A receipt must remain traceable to the invoice or financial document against which it was recorded.

BRULE-55: Receipt Void Preservation

Voiding a receipt must preserve transaction history and audit visibility.

4.13 Reporting and Audit Rules

BRULE-56: Reporting Accuracy

All reports must reflect the current valid state of the underlying business data according to the reporting context.

BRULE-57: Profit and Loss Consistency

Profit and loss reporting must be based on accurate revenue and cost figures derived from valid business records.

BRULE-58: Historical Integrity

Historical reports must preserve prior business events and should not be distorted by destructive deletion of financially relevant records.

BRULE-59: Uninvoiced Identification

Uninvoiced reporting must correctly identify valid business content that has not yet been invoiced according to the billing rules.

BRULE-60: Auditability of Financial Events

Financially significant actions such as invoice creation, voiding, receipt recording, and voucher creation must remain auditable.

4.14 Validation Message Requirements

BRULE-61: Clear Validation Feedback

When a request fails validation, the system must provide a clear and meaningful error response identifying the reason for rejection.

BRULE-62: Consistent Error Handling

Validation failures, missing references, state violations, and unauthorized access must be handled consistently across modules.

4.15 Cross-Module Consistency Rules

BRULE-63: Downstream Dependency Protection

A record must not be changed in a way that invalidates already completed downstream processing unless reversal logic exists.

BRULE-64: Operational-to-Financial Continuity

Operational records and financial records must remain connected throughout the lifecycle of a business transaction.

BRULE-65: No Contradictory States

The system must not allow a business record to exist in mutually contradictory states across related modules.

Example

- A job cannot be treated as closed for operations while remaining openly editable for dependent services without explicit business support.
- A voided invoice cannot simultaneously behave as an active collectible invoice.

4.16 Acceptance of Business Rules

The business rules in this section are considered satisfied when:

- the system enforces the defined conditions before processing transactions,

- invalid requests are rejected consistently,
- downstream financial integrity is preserved,
- state changes follow controlled business logic,
- traceability is maintained across all major modules.

5. Business Process Flows

5.1 Overview

This section defines the end-to-end business workflows supported by the LogisticsPro API. Each process describes how business operations are executed across multiple modules, including the sequence of steps, required conditions, alternate paths, and exception handling scenarios.

Each process is defined using:

- Trigger
- Preconditions
- Main Flow (Happy Path)
- Alternate Flows
- Exception Flows
- Postconditions

These flows ensure that all stakeholders understand how the system behaves across real business scenarios.

5.2 Process 1: Customer and Job Initialization

Process Name

Customer Creation and Job Card Initialization

Trigger

A new operational requirement is initiated for a customer.

Preconditions

- User is authenticated
- Customer may or may not already exist
- Required system access is available

Main Flow

1. User creates a new customer (if not existing)
2. System validates customer data
3. System stores customer record
4. User creates a new job card linked to the customer
5. System validates customer reference
6. System generates a unique Job ID
7. Job card is created with status “Open”

Alternate Flows

A1: Customer Already Exists

- User selects existing customer instead of creating a new one
- System retrieves and validates customer
- Flow continues to job creation

Exception Flows

E1: Invalid Customer Data

- System rejects request
- Error message returned

E2: Missing Customer Reference

- Job creation fails

Postconditions

- Customer record exists
- Job card is created and ready for service entries

5.3 Process 2: Add Operational Services to Job

Process Name

Operational Service Assignment

Trigger

User needs to record services related to a job

Preconditions

- Job card exists and is in Open state
- User is authenticated

Main Flow

1. User selects job card
2. User adds service item (transport/hotel/visa/misc)
3. System validates job status
4. System validates service data
5. System stores service item
6. System updates job cost

Alternate Flows

A1: Multiple Services Added

- Steps repeat for each service
- Cost recalculated dynamically

Exception Flows

E1: Job is Closed

- System rejects service addition

E2: Invalid Cost

- System rejects service

Postconditions

- Service items linked to job
- Job total cost updated

5.4 Process 3: Job Cost Finalization and Closure

Process Name

Job Finalization

Trigger

User completes operational work

Preconditions

- Job exists
- All required services recorded

Main Flow

1. User reviews job details
2. System calculates total cost
3. User initiates job closure
4. System validates job state
5. System updates job status to Closed

Alternate Flows

A1: Additional Services Required

- User adds missing services
- Flow returns to service management

Exception Flows

E1: Attempt to Close Incomplete Job

- System rejects closure

Postconditions

- Job is locked for further modification

5.5 Process 4: Batch Creation for Financial Processing

Process Name

Batch Processing

Trigger

User prepares operational items for financial processing

Preconditions

- Service items exist
- Items are eligible for batching

Main Flow

1. User selects eligible service items
2. User creates batch
3. System validates items
4. System groups items into batch

5. Batch record is created

Alternate Flows

A1: Partial Selection

- Only selected items are batched

Exception Flows

E1: Duplicate Item Selection

- System rejects batch

Postconditions

- Batch is created and ready for financial processing

5.6 Process 5: Payment Voucher Creation

Process Name

Vendor Payment Processing

Trigger

User initiates payment for vendors

Preconditions

- Batch exists
- Vendor information available

Main Flow

1. User selects batch
2. System calculates payable amount
3. User creates payment voucher
4. System validates batch

5. Voucher is created

Main Flow

1. User selects batch
2. System calculates payable amount
3. User creates payment voucher
4. System validates batch
5. Voucher is created

Postconditions

- Payment obligation recorded

5.7 Process 6: Invoice Generation

Process Name

Customer Billing

Trigger

User initiates billing for a job

Preconditions

- Job is valid
- Cost is finalized

Main Flow

1. User selects job
2. System retrieves job cost
3. User creates invoice
4. System validates job

5. Invoice is generated

Exception Flows

E1: Invalid Job

- Invoice creation fails

Postconditions

- Invoice created and ready for payment

5.8 Process 7: Receipt Processing

Process Name

Payment Recording

Trigger

Customer makes payment

Preconditions

- Invoice exists

Main Flow

1. User selects invoice
2. User records payment
3. System validates amount
4. Receipt is created

Exception Flows

E1: Overpayment

- System rejects receipt

Postconditions

- Payment recorded
- Invoice balance updated

5.9 Process 8: Reporting and Analysis

Process Name

Report Generation

Trigger

User requests report

Preconditions

- Data exists

Main Flow

1. User selects report type
2. System aggregates data
3. System generates report
4. Results returned

Postconditions

- Report available for analysis

6. Data Requirements and Data Mode

6.1 Overview

This section defines the data requirements of the LogisticsPro API, including the structure of business entities, their attributes, relationships, and constraints. It provides a conceptual data model that ensures consistency between operational workflows and financial processes.

The purpose of this section is to:

- define all core entities used in the system
- specify required data attributes
- establish relationships between entities
- define constraints and integrity rules

This section serves as the foundation for database design, API request/response structures, and data validation.

6.2 Data Entity Classification

The system data is categorized into the following groups:

1. **Master Data**
    - Customer
    - Vendor
    - Event
2. **Operational Data**
    - Job Card
    - Service Items (Transport, Hotel, Visa, Miscellaneous)
3. **Financial Data**
    - Batch
    - Payment Voucher
    - Invoice
    - Proforma Invoice
    - Receipt

6.3 Entity Definitions

6.3.1 Customer Entity

Description

Represents a business customer receiving services.

Attributes

Field	Type	Mandatory	Description
CustomerId	Unique Identifier	Yes	Primary key
Name	String	Yes	Customer name
ContactNumber	String	No	Contact details
Email	String	No	Email address
Status	Enum	Yes	Active / Inactive

Constraints

- CustomerId must be unique
- Name must not be empty
- Inactive customers should not be used for new jobs

6.3.2 Vendor Entity

Description

Represents service providers such as transport, hotel, or visa vendors.

Attributes

Field	Type	Mandatory	Description
VendorId	Unique Identifier	Yes	Primary key
Name	String	Yes	Vendor name

ServiceType	Enum	Yes	Type of service
Status	Enum	Yes	Active / Inactive

Constraints

- VendorId must be unique
- ServiceType must be valid

6.3.3 Event Entity

Description

Represents a grouping context for jobs.

Attributes

Field	Type	Mandatory	Description
EventId	Unique Identifier	Yes	Primary key
EventName	String	Yes	Name of event
StartDate	Date	Yes	Event start
EndDate	Date	Yes	Event end

Constraints

- EndDate ≥ StartDate

6.3.4 Job Card Entity

Description

Represents a unit of operational work.

Attributes

Field	Type	Mandatory	Description
JobId	Unique Identifier	Yes	Primary key
CustomerId	Reference	Yes	Linked customer
EventId	Reference	No	Linked event

Status	Enum	Yes	Open / Closed
CreatedDate	DateTime	Yes	Creation timestamp
TotalCost	Decimal	Yes	Calculated value

Constraints

- Must reference valid Customer
- Cannot be modified when Closed

6.3.5 Service Item Entity (Generic)

Description

Represents operational services linked to a job.

Attributes

Field	Type	Mandatory	Description
ServiceId	Unique Identifier	Yes	Primary key
JobId	Reference	Yes	Linked job
VendorId	Reference	No	Linked vendor
ServiceType	Enum	Yes	Transport / Hotel / Visa / Misc
Cost	Decimal	Yes	Service cost

6.3.5.1 Transportation Service

Field	Type	Mandatory
PickupLocation	String	Yes
DropLocation	String	Yes
TravelDate	Date	Yes

6.3.5.2 Hotel Service

Field	Type	Mandatory
CheckInDate	Date	Yes

CheckOutDate	Date	Yes
RoomCount	Integer	Yes

6.3.5.3 Visa Service

Field	**Type**	**Mandatory**
VisaType	String	Yes
ProcessingDate	Date	Yes

6.3.5.4 Miscellaneous Service

Field	**Type**	**Mandatory**
Description	String	Yes

Constraints (All Services)

- Cost ≥ 0
- Must reference valid Job
- Cannot exist independently

6.3.6 Batch Entity

Description

Represents grouping of service items.

Attributes

Field	Type	Mandatory
BatchId	Unique Identifier	Yes
CreatedDate	DateTime	Yes
Status	Enum	Yes

6.3.7 Payment Voucher Entity

Attributes

Field	Type	Mandatory
VoucherId	Unique Identifier	Yes
BatchId	Reference	Yes
VendorId	Reference	Yes
Amount	Decimal	Yes

6.3.8 Invoice Entity

Attributes

Field	Type	Mandatory
InvoiceId	Unique Identifier	Yes
JobId	Reference	Yes
TotalAmount	Decimal	Yes
Status	Enum	Yes

6.3.9 Proforma Invoice Entity

Similar to Invoice but preliminary.

6.3.10 Receipt Entity

Attributes

Field	Type	Mandatory
ReceiptId	Unique Identifier	Yes
InvoiceId	Reference	Yes
Amount	Decimal	Yes

6.4 Entity Relationships

Core Relationships

- Customer → Job Card (1:N)
- Event → Job Card (1:N)
- Job Card → Service Items (1:N)
- Vendor → Service Items (1:N)
- Job Card → Invoice (1:N)
- Invoice → Receipt (1:N)
- Batch → Service Items (1:N)
- Batch → Payment Voucher (1:N)

6.5 Data Integrity Rules

- Foreign keys must reference valid records
- Cascading deletes must be controlled
- Financial records must not be orphaned
- Data must remain consistent across modules

6.6 Data Lifecycle Considerations

- Master data may be deactivated instead of deleted
- Operational data evolves until job closure
- Financial data must remain immutable after finalization
- Historical data must be preserved

7. Non-Functional Requirements

7.1 Overview

This section defines the non-functional requirements for the LogisticsPro API. While functional requirements describe the business operations the system must perform, non-functional requirements define the quality attributes the system must satisfy during operation. These requirements influence architectural decisions, implementation standards, infrastructure planning, security controls, operational support, and long-term maintainability.

The LogisticsPro API supports operational and financial business processes. Because of this, non-functional requirements are critical. Failures in performance, security, availability, or data consistency can directly affect billing accuracy, operational continuity, reporting reliability, and audit readiness. Therefore, the system must not only implement the required business functions, but must also do so in a stable, secure, and controlled manner.

The non-functional requirements in this document are grouped into the following categories:

- Performance requirements
- Availability requirements
- Reliability requirements
- Security requirements
- Data integrity requirements
- Scalability requirements
- Maintainability requirements
- Auditability and traceability requirements
- Usability considerations for API consumers
- Compatibility and interoperability requirements

7.2 Performance Requirements

7.2.1 General Performance Expectation

The LogisticsPro API must provide responsive service for standard business operations under normal expected load conditions. Common read and write operations such as retrieving master data, creating job cards, adding service items, and generating financial records must complete within an acceptable response time for internal business use.

7.2.2 Response Time

The system should support the following response expectations under normal operating conditions:

- simple retrieval operations should return responses within a short, acceptable time window
- standard create and update operations should complete without noticeable delay for end users or client systems
- report generation operations may require longer execution time than transactional endpoints, but must still complete within a controlled and predictable duration
- long-running or aggregation-heavy operations must be designed to avoid unnecessary blocking of unrelated transactional processes

Where exact performance targets are later agreed at solution design stage, those targets must align with business criticality and expected load.

7.2.3 Concurrent Usage

The system must support concurrent API requests from multiple authenticated users or client applications without causing data corruption, inconsistent record states, or unacceptable performance degradation.

7.2.4 Efficient Data Access

The API must retrieve and persist data efficiently. Query patterns, indexing strategies, and data relationships must be designed to avoid unnecessary load, repeated expensive queries, or poor performance under growing operational data volume.

7.2.5 Reporting Performance

Operational and financial reports must be generated in a manner that balances completeness and performance. Reporting requests should not cause excessive degradation of transactional business operations.

7.3 Availability Requirements

7.3.1 Service Availability

The LogisticsPro API must be available during defined business operating periods and should support reliable access for authorized users and dependent systems.

7.3.2 Controlled Downtime

Scheduled maintenance, deployments, or configuration changes must be planned and communicated to minimize disruption to business users and downstream processes.

7.3.3 Recovery from Failure

In the event of unexpected service interruption, the system must support recovery procedures that restore service with minimal data loss and minimal impact on in-progress business operations.

7.3.4 Dependency Awareness

Availability expectations must consider the availability of core dependencies such as database access, authentication configuration, infrastructure services, and network connectivity.

7.4 Reliability Requirements

7.4.1 Transaction Reliability

Business transactions must be processed reliably so that each valid request produces a predictable and consistent result.

7.4.2 Fault Handling

The system must handle invalid inputs, unavailable dependencies, processing conflicts, and unexpected runtime errors in a controlled manner. Failures must not leave business records in an inconsistent or partially committed state.

7.4.3 Stable Financial Processing

Financial transactions such as invoice creation, receipt processing, voucher creation, and void operations must be especially reliable because they affect auditability, business reporting, and financial correctness.

7.4.4 Idempotency Consideration

Where duplicate client submissions are possible, particularly in create or financial operations, the implementation should consider protections against accidental duplicate processing.

7.4.5 Consistent Operation Under Load

The system must continue to behave correctly when multiple users perform related operational or financial activities at the same time.

7.5 Security Requirements

7.5.1 Authenticated Access

Protected endpoints must require valid authentication before any business processing is performed.

7.5.2 Authorization Enforcement

Where role-based or permission-based access applies, the system must ensure that users can access only the resources and operations they are permitted to perform.

7.5.3 Secure Token Handling

Authentication tokens must be validated securely, including checks for signature validity, expiration, and expected issuer or audience where applicable.

7.5.4 Data Protection

Sensitive business data must be protected in transit and, where required, at rest. The system must prevent unauthorized disclosure of customer, vendor, operational, or financial information.

7.5.5 Input Validation

All API inputs must be validated to reduce the risk of malformed requests, invalid business operations, injection-style attacks, or unintended system behavior.

7.5.6 Error Message Control

Error responses must not expose sensitive internal implementation details such as stack traces, raw SQL errors, secret values, or infrastructure-specific information to unauthorized users or client applications.

7.5.7 Audit-Sensitive Financial Security

Financial endpoints must be subject to stricter control because incorrect access or misuse could affect billing, payments, receipts, and reports.

7.5.8 Secure Configuration

Secrets, connection strings, signing keys, and environment-specific configurations must be managed securely and must not be hardcoded in a way that compromises system security.

7.6 Data Integrity Requirements

7.6.1 Referential Integrity

All relationships between customers, vendors, events, job cards, service items, batches, invoices, receipts, and vouchers must remain valid throughout the record lifecycle.

7.6.2 Transactional Consistency

The system must preserve consistency across multi-step operations. If part of a transaction fails, the system must prevent incomplete or contradictory business outcomes.

7.6.3 Financial Integrity

Amounts used in invoices, receipts, payment vouchers, and reporting must accurately reflect the source business records from which they are derived.

7.6.4 Prevention of Orphan Records

Deletion, deactivation, or update logic must not leave child or related records in a broken or untraceable state.

7.6.5 Controlled State Changes

Status changes must follow the defined business process. The system must prevent contradictory states across modules.

7.6.6 Historical Preservation

Data required for historical reporting, auditing, or business review must be preserved even when records are voided, closed, or deactivated.

7.7 Scalability Requirements

7.7.1 Growth in Transaction Volume

The system should support increasing numbers of customers, vendors, jobs, services, invoices, and receipts without requiring major redesign of the core business model.

7.7.2 Growth in Data Volume

The data model and data access patterns must remain manageable as historical operational and financial records accumulate over time.

7.7.3 Module Expansion

The architecture should allow future expansion of business modules, service types, or financial features without destabilizing the existing core functions.

7.7.4 Reporting Scalability

Reporting mechanisms should continue to operate effectively as historical data size grows, potentially requiring optimized queries, summaries, or archival strategies.

7.8 Maintainability Requirements

7.8.1 Modular Design

The API should be designed in a modular way so that business modules such as jobs, services, billing, and reporting can be maintained and extended independently where practical.

7.8.2 Readable and Structured Code

Implementation should follow clear coding standards, consistent naming, and structured separation of concerns to simplify maintenance and onboarding of new developers.

7.8.3 Business Rule Maintainability

Business rules should be implemented in a way that allows controlled enhancement without requiring high-risk changes across unrelated modules.

7.8.4 Configurability

Where environment-specific settings or operational thresholds exist, they should be configurable rather than embedded directly into business logic.

7.8.5 Test Support

The system should be structured in a way that supports verification of core business logic, service flows, and financial calculations during development and maintenance.

7.9 Auditability and Traceability Requirements

7.9.1 Financial Audit Support

Financially significant actions must remain traceable. This includes invoice creation, voucher creation, receipt recording, void actions, and related downstream changes.

7.9.2 Historical Visibility

The system must preserve sufficient historical information to support review of past business activity, financial reconciliation, and management reporting.

7.9.3 User Action Traceability

Where applicable, the system should be able to associate key actions with the authenticated user or process that performed them.

7.9.4 Non-Destructive Financial Reversal

Voiding or reversing financial records should preserve record history rather than permanently removing evidence of the original transaction.

7.10 API Usability Requirements

7.10.1 Consistent Request and Response Structure

API endpoints should follow consistent conventions for request payloads, response payloads, status codes, and error formats so that client systems can integrate reliably.

7.10.2 Clear Error Responses

When a request fails, the API must provide meaningful and consistent error messages that help client applications understand the cause of rejection without exposing sensitive internal details.

7.10.3 Predictable Endpoint Behavior

Endpoints must behave consistently according to their documented purpose. Similar operations across modules should follow similar usage patterns where possible.

7.10.4 Documentation Readiness

The API should be suitable for formal documentation through mechanisms such as Swagger or equivalent interface documentation so that consumers can understand available endpoints, required inputs, and expected outputs.

7.11 Compatibility and Interoperability Requirements

7.11.1 Standard API Communication

The system must support standard API communication patterns suitable for integration with web clients, internal systems, and authorized third-party consumers where applicable.

7.11.2 Data Exchange Compatibility

Payload structures should use common and interoperable formats so that the system can exchange data reliably with approved consuming applications.

7.11.3 Environment Portability

The API should be deployable across supported development, testing, staging, and production environments without requiring major code changes to business logic.

7.12 Logging and Monitoring Requirements

7.12.1 Operational Logging

The system should maintain logs for significant operational and financial events, application failures, validation issues, and security-related access failures.

7.12.2 Error Monitoring

Unexpected failures must be detectable through monitoring, alerting, or log review mechanisms so that support teams can identify and respond to production issues promptly.

7.12.3 Business Event Visibility

Where appropriate, logging should provide sufficient visibility into major business operations such as job closure, invoice issuance, receipt processing, and voucher creation.

7.12.4 Sensitive Data Protection in Logs

Logs must not expose sensitive data in an unsafe manner. Masking or exclusion should be applied where necessary.

7.13 Backup and Recovery Requirements

7.13.1 Data Backup

Business-critical data must be backed up according to the organization’s recovery requirements.

7.13.2 Recovery Capability

The system must support restoration of critical business data in the event of database corruption, infrastructure failure, or accidental loss.

7.13.3 Recovery Integrity

Recovery processes must restore records in a way that preserves operational and financial consistency.

7.14 Compliance and Governance Considerations

7.14.1 Controlled Financial Behavior

Because the system includes invoices, receipts, vouchers, and profit reporting, implementation must preserve financial governance principles such as accuracy, traceability, and controlled reversal.

7.14.2 Data Handling Responsibility

Business data must be handled in accordance with applicable organizational standards for confidentiality, retention, and access control.

7.14.3 Change Control

Changes to business-critical logic, especially financial calculations and lifecycle rules, should be managed through controlled review and deployment processes.

7.15 Non-Functional Acceptance Criteria

The non-functional requirements are considered satisfied when the system demonstrates that:

- authenticated access is enforced consistently across protected endpoints
- operational and financial data remain reliable and internally consistent
- standard business operations perform within acceptable response expectations
- the API remains stable under normal concurrent usage
- meaningful logging, error handling, and monitoring are available

- financially significant actions remain traceable and auditable
- the system can be maintained and extended without destabilizing core business processes

8. API Requirements and Interface Requirements

8.1 Overview

This section defines the interface-level expectations for the LogisticsPro API. Since the system is designed as a backend API, it is not sufficient to describe only the business processes and data requirements. The document must also define how external or internal client applications are expected to interact with the system, how requests are submitted, how responses are returned, how errors are communicated, and how interface consistency is maintained across modules.

The purpose of this section is to establish a common contract for API design and implementation. It ensures that developers, integrators, testers, and support teams understand the expected behavior of the interface and the rules that govern communication between the API and consuming systems.

The API requirements in this section focus on behavior and standards rather than low-level technical implementation details. They define how the API should behave from a consumer perspective and how interface consistency must be preserved across all modules.

8.2 API Design Principles

The LogisticsPro API shall be designed as a REST-oriented service interface that exposes business capabilities through clearly structured endpoints. The API must support consistent interaction patterns across all business domains, including master data management, job processing, operational services, financial processing, and reporting.

The following design principles shall apply:

- the API shall expose business operations in a clear and predictable manner
- similar operations across different modules shall follow similar naming and behavioral patterns

- the API shall use standard HTTP methods according to the nature of the operation
- the API shall return consistent response structures wherever practical
- the API shall provide meaningful status codes and error information
- the API shall protect business-critical operations using authentication and authorization controls
- the API shall preserve backward compatibility where possible when enhancements are introduced

These principles are intended to improve maintainability, integration quality, and long-term usability of the API.

8.3 Interface Consumers

The LogisticsPro API may be consumed by the following types of clients:

8.3.1 Internal Business Applications

Internal web or desktop applications used by operations, administration, and finance teams may consume the API to perform daily business tasks such as customer management, job updates, invoice generation, and reporting.

8.3.2 Administrative Tools

Back-office or support tools may consume the API for maintenance, monitoring, troubleshooting, or controlled data correction activities.

8.3.3 Reporting or Integration Services

Approved internal services or downstream systems may consume the API, or related data it exposes, to support reporting, analytics, or controlled business integrations.

8.3.4 Future External Consumers

Where business needs evolve, selected external or third-party systems may be granted controlled access through approved authentication and interface governance mechanisms.

8.4 Communication Protocol Requirements

The API shall support standard HTTP-based communication using structured request and response payloads.

Requirements

- requests shall be submitted over secure network communication channels
- request and response payloads shall use a standard machine-readable format
- endpoints shall be accessible using predictable URI structures
- the API shall support standard HTTP verbs appropriate to the requested action

The API is expected to operate as a stateless service for business requests, with each request containing the information required for authentication, authorization, and processing.

8.5 HTTP Method Usage

The API shall use HTTP methods consistently according to operation type.

GET

Used for retrieval of data without modifying server-side state.

POST

Used for creation of new records or for business operations that initiate controlled processing.

PUT / PATCH

Used for updating existing data, depending on the update strategy adopted by the implementation.

DELETE

Used for removal operations only where deletion is allowed by business rules. Where historical preservation is required, logical deletion or deactivation may be preferred over physical deletion.

Requirements

- unsafe or state-changing operations shall not be implemented through retrieval-only methods
- operations that create or materially alter business state shall use appropriate state-changing methods
- endpoint behavior shall align with the intended semantics of the HTTP method used

8.6 Resource Structure Requirements

The LogisticsPro API shall organize business functionality around clearly defined resources representing key domain entities.

Typical Resource Areas

- authentication
- customers
- vendors
- events
- job cards
- transportation services
- hotel services
- visa services
- miscellaneous services
- batches
- payment vouchers
- invoices
- proforma invoices
- receipts

- reports

Requirements

- each resource area shall have a clearly defined business purpose
- resource naming shall be consistent and understandable
- sub-resources or action-based endpoints shall be used only where necessary to reflect business meaning
- resource structures shall preserve clarity between operational and financial modules

8.7 Request Requirements

8.7.1 Request Structure

Each API request shall contain all information necessary for the system to validate and process the operation.

Requirements

- required fields shall be supplied for all create and update requests
- field names shall follow consistent naming conventions
- requests shall support only valid business attributes relevant to the target operation
- unexpected or unsupported payload structures should be rejected or safely ignored according to the defined validation strategy

8.7.2 Identifier Usage

Where an operation targets an existing record, the request shall identify that record using the appropriate identifier.

Requirements

- identifiers shall uniquely identify the target resource
- invalid or non-existent identifiers shall result in an appropriate error response
- identifiers must not be ambiguous across business domains

8.7.3 Filtering and Query Parameters

Where supported, retrieval endpoints may accept filters, search values, sorting options, paging controls, or date ranges.

Requirements

- query parameters shall be validated
- invalid filter combinations shall be rejected or handled consistently
- filtering shall not produce misleading or incomplete results without indication

8.8 Status Code Requirements

The API shall use HTTP status codes consistently to reflect the outcome of request processing.

Expected Status Categories

- success status codes for completed operations
- client error status codes for validation issues, missing resources, unauthorized access, or invalid business actions
- server error status codes for unexpected internal failures

Requirements

- successful retrieval operations shall return an appropriate success code
- successful creation operations shall return a creation-related status where applicable
- validation failures shall return a client error status
- unauthorized or unauthenticated requests shall return appropriate security-related status codes
- not-found conditions shall return an appropriate missing-resource status
- unexpected processing failures shall return a server error status without exposing sensitive internal details

Consistency Requirement

The same type of business outcome shall not return conflicting status code behavior across different modules without a justified reason.

8.9 Authentication Interface Requirements

8.9.1 Authentication Endpoint Behavior

The API shall provide a secure mechanism for obtaining an authentication token or equivalent authorized session artifact.

Requirements

- authentication requests shall validate submitted credentials
- successful authentication shall return a token or equivalent access artifact
- failed authentication shall return a clear but secure rejection response
- authentication failures shall not leak sensitive security details

8.9.2 Authenticated Request Behavior

Protected endpoints shall require the caller to present valid authentication credentials with the request.

Requirements

- missing credentials shall result in request rejection
- invalid credentials shall result in request rejection
- expired credentials shall result in request rejection
- authorized requests shall proceed only after successful validation

8.10 Authorization Interface Requirements

Where role-based or permission-based access rules apply, the API shall enforce them consistently.

Requirements

- users shall not be able to perform restricted actions without sufficient permission
- access to financially sensitive operations may require stricter authorization than general retrieval operations
- authorization failures shall return a controlled error response

8.11 Validation and Error Response Requirements

8.11.1 Input Validation

The API shall validate all incoming requests before business processing begins.

Requirements

- missing required fields shall cause validation failure
- malformed field values shall cause validation failure
- invalid references to related entities shall cause validation failure
- business rule violations shall cause request rejection

8.11.2 Error Response Structure

Error responses shall be meaningful, structured, and consistent.

Requirements

- the response shall indicate that the request failed
- the response shall identify the general reason for failure
- validation errors should clearly identify affected fields or conditions where practical
- the response shall avoid exposing internal implementation details, stack traces, or sensitive data

8.11.3 Business Rule Violation Handling

Where a request fails due to business state or rule restrictions, the response shall communicate the nature of the rule violation in a controlled manner.

Examples

- attempting to modify a closed job
- attempting to create duplicate financial processing
- attempting to overpay an invoice
- attempting to reference an invalid downstream state

8.12 Pagination, Sorting, and Search Requirements

Where list endpoints return potentially large result sets, the API should support structured retrieval controls.

Requirements

- list endpoints should support paging where the result size may become large
- sorting behavior, where supported, shall be documented and predictable
- search and filtering inputs shall be validated
- paging metadata should be returned where needed to support client navigation

This is especially relevant for:

- customer lists
- vendor lists
- job lists
- invoice lists
- receipt lists
- report-oriented retrieval operations

8.13 Date, Time, and Numeric Interface Requirements

The API shall handle date, time, and numeric values consistently across all endpoints.

Requirements

- date and time formats shall follow a consistent standard
- financial values shall use an appropriate numeric format and precision
- request payloads with invalid or ambiguous date or amount formats shall be rejected
- time-related fields shall be interpreted consistently across environments

This is especially important for:

- event dates
- hotel booking dates
- visa processing dates
- created and updated timestamps
- invoice totals
- receipt amounts
- voucher values

8.14 Financial Endpoint Requirements

Financial endpoints require stricter interface behavior because they affect invoices, receipts, vouchers, and reporting.

Requirements

- financial create and update operations shall apply stricter validation than general master data endpoints
- financial responses shall clearly indicate financial state, amount, and processing outcome
- financially significant operations such as voiding shall return a clear confirmation result
- financial endpoints shall maintain interface behavior that supports audit and review activities

8.15 Reporting Interface Requirements

Reporting endpoints may differ from standard transactional endpoints due to aggregation and filtering needs.

Requirements

- reporting endpoints shall support relevant filters such as date range, status, customer, or other business dimensions where required
- report results shall reflect the latest valid business data at the time of generation
- reporting interfaces shall clearly distinguish between operational reports and financial reports
- long-running reports should be handled in a controlled manner that does not degrade overall API usability

8.16 Versioning Requirements

The API should support a versioning strategy that allows enhancement without unnecessarily breaking existing consumers.

Requirements

- public or shared endpoints should follow a controlled versioning approach
- breaking interface changes must be managed carefully and communicated appropriately
- future enhancements should preserve compatibility where practical

8.17 Documentation Requirements

The API shall be documented in a way that allows consumers to understand available operations and their expected inputs and outputs.

Requirements

- each endpoint shall have a clear business description
- request fields shall be documented

- response fields shall be documented
- authentication requirements shall be documented
- expected status codes and common error conditions shall be documented
- API documentation should be available in a format suitable for developer consumption

8.18 Interface Logging and Traceability Requirements

Requirements

- requests to business-critical endpoints should be traceable for support and audit purposes
- failures in interface processing should be logged consistently
- financial and security-sensitive operations should have stronger trace visibility
- request correlation support should be considered for troubleshooting across services or layered components

8.19 API Acceptance Criteria

The API and interface requirements are considered satisfied when:

- the API exposes clearly structured and consistently named business endpoints
- request and response behavior is predictable across modules
- authentication and authorization are enforced correctly
- validation failures and business rule violations return controlled, meaningful responses
- status code usage is consistent with business outcomes
- list and report endpoints support practical retrieval behavior
- financial operations apply stronger controls and clearer traceability
- the interface can be understood and consumed reliably by approved client systems

9. Assumptions, Dependencies, Risks, and Exception Handling

9.1 Overview

This section documents the key assumptions, dependencies, operational risks, and exception-handling expectations for the LogisticsPro API. These factors influence how the system is designed, implemented, deployed, and maintained. They also provide important context for decision-making during development and support future change management activities.

Assumptions describe conditions considered true for the purpose of system design and implementation. Dependencies identify external or internal elements the system relies on for correct operation. Risks describe events or conditions that could negatively affect system behavior, business continuity, or data integrity. Exception handling requirements define how the system should respond when expected processing cannot be completed successfully.

Because the LogisticsPro API supports operational and financial workflows, failures or incorrect assumptions can create business disruption, billing issues, reporting inaccuracies, and audit concerns. For this reason, these items must be explicitly recognized rather than left implied.

9.2 Assumptions

The following assumptions apply to the LogisticsPro API and to the scope of this document.

9.2.1 Business User Assumptions

- the API will be consumed primarily by authorized internal users or trusted client systems
- users interacting with the system will have sufficient business knowledge to understand the meaning of customers, vendors, events, jobs, services, invoices, receipts, and related financial records
- users are expected to follow the intended process sequence and not bypass key business steps through unsupported manual actions

9.2.2 Data Assumptions

- customer, vendor, and event data will be maintained accurately and kept reasonably up to date
- referenced master data will exist before downstream records that depend on it are created
- financial amounts provided or derived by the system will use a consistent business currency model unless multi-currency support is explicitly introduced later
- business records entered through the system represent valid real-world operational activity

9.2.3 Technical Assumptions

- the API will operate in an environment where secure authentication configuration is available
- the database platform and infrastructure required by the API will be provisioned and maintained appropriately
- network connectivity between the API and its required dependencies will be sufficiently stable for business use
- client applications will submit data in the formats and structures expected by the API

9.2.4 Operational Assumptions

- business users will use supported API consumers rather than interacting directly with the database
- financial operations such as invoice, voucher, and receipt processing will follow approved business procedures
- voiding, corrections, and other sensitive actions will be performed only by authorized users or systems

9.3 Dependencies

The LogisticsPro API depends on several supporting components and conditions in order to function correctly.

9.3.1 Authentication Dependency

The system depends on the configured authentication mechanism, including token issuance and validation behavior, to protect business operations.

Dependency Impact:
If authentication is misconfigured or unavailable, protected business operations may become inaccessible or insecure.

9.3.2 Database Dependency

The API depends on a functioning database layer for storage and retrieval of master data, operational data, financial data, and reporting inputs.

Dependency Impact:
Database unavailability, corruption, performance issues, or schema inconsistencies may prevent core operations and financial processing.

9.3.3 Configuration Dependency

The system depends on correct environment-specific configuration for items such as database connections, authentication settings, API behavior settings, and operational controls.

Dependency Impact:
Incorrect configuration may cause runtime failures, security weaknesses, or incorrect business behavior.

9.3.4 Infrastructure Dependency

The API depends on hosting, network, runtime, logging, and monitoring infrastructure.

Dependency Impact:
Infrastructure instability may reduce availability, impair performance, or hinder diagnosis of incidents.

9.3.5 Data Quality Dependency

The correctness of downstream processes depends on the quality of upstream master and operational data.

Dependency Impact:
Poor customer, vendor, event, or service data can lead to billing problems, incorrect reports, and broken business traceability.

9.3.6 Business Process Dependency

Certain modules depend on prior completion of earlier business steps.

Examples

- job creation depends on valid customer data
- service entry depends on valid job data
- batch processing depends on eligible operational items
- invoice creation depends on valid billable content
- receipt processing depends on a valid invoice or receivable source

Dependency Impact:
Skipping required earlier steps can cause invalid downstream processing or rejected transactions.

9.4 Risks

The following risks are relevant to the implementation and operation of the LogisticsPro API.

9.4.1 Data Integrity Risk

There is a risk that incorrect updates, deletions, or partial failures may create inconsistent relationships between operational and financial records.

Examples

- service items no longer matching job totals
- invoices no longer matching billable content
- receipts applied incorrectly
- vendor vouchers not matching underlying batch items

Impact

- incorrect reporting
- billing disputes
- audit concerns
- operational confusion

Mitigation

- strict referential integrity
- transactional processing for multi-step changes
- controlled modification rules
- validation of downstream dependencies before change

9.4.2 Financial Accuracy Risk

There is a risk that invoice totals, voucher amounts, or receipt values may become inaccurate due to incorrect calculations, duplicate processing, or inconsistent business logic.

Impact

- revenue loss
- overbilling or underbilling
- payment disputes
- unreliable P&L reporting

Mitigation

- centralized financial calculation logic
- duplicate prevention controls
- reconciliation checks
- strong validation before financial finalization

9.4.3 Unauthorized Access Risk

There is a risk that sensitive operational or financial actions may be exposed to users without appropriate permission if authentication or authorization is not correctly enforced.

Impact

- unauthorized data access
- improper financial actions
- data leakage
- governance and compliance issues

Mitigation

- secure authentication enforcement
- strict authorization controls
- secure configuration management
- access logging and review

9.4.4 Duplicate Processing Risk

There is a risk of duplicate creation of jobs, service items, invoices, receipts, vouchers, or batches due to repeated user submissions, integration retries, or weak duplicate controls.

Impact

- duplicate billing
- duplicate payment obligations

- inconsistent reporting
- manual cleanup effort

Mitigation

- idempotency-aware design where appropriate
- duplicate business checks
- transaction safeguards
- user feedback on completed processing

9.4.5 Concurrency Risk

There is a risk that concurrent updates by multiple users or processes could cause conflicting changes or inconsistent state.

Examples

- one user closing a job while another adds a service
- simultaneous receipt creation against the same invoice
- multiple processes attempting to batch or invoice the same items

Impact

- inconsistent business state
- incorrect totals
- process conflicts
- support incidents

Mitigation

- concurrency-aware update strategy
- controlled state validation
- transactional consistency

- locking or optimistic concurrency where appropriate

9.4.6 Reporting Accuracy Risk

There is a risk that reports may become misleading if they use incomplete, stale, invalid, or partially processed data.

Impact

- poor business decisions
- incorrect profitability view
- financial reconciliation issues

Mitigation

- consistent reporting rules
- validated reporting sources
- clear treatment of voided, inactive, and historical records
- periodic reconciliation review

9.4.7 Availability and Dependency Failure Risk

There is a risk that database, infrastructure, configuration, or network failures may interrupt business operations.

Impact

- business downtime
- delayed invoicing
- inability to process operations
- user frustration

Mitigation

- monitoring and alerting

- backup and recovery planning
- resilient deployment practices
- operational support procedures

9.4.8 Change Impact Risk

There is a risk that changes to one module may unintentionally affect another because of the strong dependency between operational and financial workflows.

Impact

- regression defects
- broken downstream processing
- data inconsistencies after release

Mitigation

- impact analysis for changes
- clear requirement traceability
- controlled release practices
- validation of cross-module behavior

9.5 Exception Handling Requirements

The system must respond to exceptional conditions in a controlled and consistent manner. Exception handling must preserve data integrity, avoid misleading outcomes, and provide useful but safe feedback to API consumers.

9.5.1 General Exception Handling Principle

When an operation cannot be completed successfully, the system must fail in a controlled manner and must not leave records in a contradictory or partially committed state.

9.5.2 Validation Exceptions

Where request data is missing, malformed, incomplete, or violates business rules, the system must reject the request before performing business processing.

Examples

- missing mandatory fields
- invalid dates
- negative financial values
- invalid entity references
- attempts to modify restricted records

Required Behavior

- reject request
- return clear validation or business-rule error response
- do not partially persist invalid data

9.5.2 Validation Exceptions

Where request data is missing, malformed, incomplete, or violates business rules, the system must reject the request before performing business processing.

Examples

- missing mandatory fields
- invalid dates
- negative financial values
- invalid entity references
- attempts to modify restricted records

Required Behavior

- reject request
- return clear validation or business-rule error response
- do not partially persist invalid data

9.5.4 Not Found Exceptions

Where a requested target resource does not exist, the system must return a clear not-found result rather than behave as though the action succeeded.

Examples

- customer not found
- job card not found
- invoice not found
- receipt target not found

9.5.4 Not Found Exceptions

Where a requested target resource does not exist, the system must return a clear not-found result rather than behave as though the action succeeded.

Examples

- customer not found
- job card not found
- invoice not found
- receipt target not found

9.5.6 Duplicate Processing Exceptions

Where the system detects that the same business content is being processed again in a conflicting way, the request must be rejected or otherwise handled according to the duplication policy.

Examples

- duplicate invoice generation
- duplicate voucher creation
- duplicate receipt registration for the same transaction intent

9.5.7 Dependency Failure Exceptions

Where a required dependency such as the database or supporting service is unavailable, the system must fail safely.

Required Behavior

- avoid partial commit where possible
- return controlled failure response
- log the failure for support investigation
- allow recovery procedures where applicable

9.5.8 Unexpected Internal Exceptions

Where an unforeseen system error occurs, the API must prevent sensitive internal details from being exposed while still ensuring that the failure is recorded for diagnosis.

Required Behavior

- return generic internal error response
- log diagnostic details securely
- preserve transaction integrity

9.6 Exception Handling Principles by Business Area

9.6.1 Master Data Exceptions

Failures in customer, vendor, or event processing must not break downstream integrity. Invalid creates or updates must be rejected before dependent records are affected.

9.6.2 Operational Exceptions

Failures during job or service processing must not result in incorrect cost totals, invalid state changes, or partial operational records.

9.6.3 Financial Exceptions

Failures during invoice, voucher, receipt, or batch processing must be handled with stricter integrity controls due to their financial significance.

9.6.4 Reporting Exceptions

If report generation fails, the system must return a controlled response and must not return misleading or partially generated financial conclusions as though they were complete.

9.7 Risk Monitoring and Review

Risks and exception patterns should be monitored throughout the lifecycle of the system.

Requirements

- production issues affecting financial integrity should be treated as high priority
- repeated validation and business-rule failures should be analyzed for process improvement opportunities
- access failures and unusual request patterns should be monitored for security concerns
- changes to core modules should trigger review of related downstream risks

9.8 Acceptance Criteria for Assumptions, Dependencies, Risks, and Exception Handling

This section is considered satisfied when:

- major assumptions are explicitly documented and reviewed,
- critical dependencies are identified and understood,

- key operational and financial risks are recognized with mitigation direction,
- the system defines controlled behavior for validation, security, state, dependency, and internal failure conditions,
- exceptional processing does not leave the system in an inconsistent state.

10. Requirements Traceability Matrix (RTM)

10.1 Overview

The Requirements Traceability Matrix (RTM) provides a structured mapping between business requirements, functional requirements, and business rules. Its purpose is to ensure that all business needs are fully addressed by the system design and that every functional component can be traced back to a clearly defined business objective.

The RTM supports:

- completeness validation (no missing requirements)
- impact analysis (understanding effect of changes)
- testing alignment (ensuring all requirements are verifiable)
- stakeholder review and approval

A well-defined RTM ensures that the LogisticsPro API is not implemented based on isolated requirements, but rather as a coherent system aligned with business goals.

10.2 Traceability Structure

The RTM maps the following:

- **Business Requirement (BR)** → What the business needs
- **Functional Requirement (FR)** → How the system fulfills it
- **Business Rules (BRL)** → Constraints governing behavior

10.3 Traceability Matrix

Business Requirement	Description	Functional Requirement(s)	Business Rules
BR-01	Manage customer data	FR-03, FR-04, FR-05	BRL-01, BRL-02
BR-02	Manage vendor data	FR-06, FR-07	BRL-03
BR-03	Manage events	FR-08	BRL-04
BR-04	Create and manage job cards	FR-09, FR-10, FR-11	BRL-05, BRL-06
BR-05	Add and manage services	FR-12, FR-13, FR-14	BRL-07, BRL-08
BR-06	Calculate job cost	FR-15	BRL-09
BR-07	Group services into batches	FR-16	BRL-10
BR-08	Manage vendor payments	FR-17	BRL-11
BR-09	Generate invoices	FR-18, FR-19	BRL-12, BRL-13
BR-10	Record customer payments	FR-20	BRL-14
BR-11	Generate reports	FR-21	BRL-15
BR-12	Secure system access	FR-01, FR-02	BRL-16

10.4 Traceability Coverage Validation

To ensure completeness, the following checks must be satisfied:

- every business requirement is mapped to at least one functional requirement
- every functional requirement supports at least one business requirement
- every critical business rule is linked to relevant functional behavior
- no orphan requirements exist without traceability

10.5 Impact Analysis Usage

The RTM enables impact analysis when changes occur.

Example

If a change is requested for:

- **Invoice logic (FR-18)**

Then impacted areas include:

- BR-09 (billing)
- BRL-12 (invoice validation rules)
- Payment processing (FR-20)
- Reporting outputs

This prevents isolated changes that break the system.

10.6 Testing Alignment

The RTM supports testing by ensuring:

- each functional requirement can be validated
- business rules are testable
- end-to-end business flows are covered

Example:

- FR-11 (Close Job)
  → Test must verify job cannot be updated after closure
  → Linked to BRL-06

10.7 Maintenance and Change Control

The RTM must be updated when:

- new business requirements are introduced

- functional behavior changes
- business rules are modified
- modules are added or removed

Failure to update the RTM will result in:

- broken traceability
- hidden defects
- inconsistent implementation

10.8 Traceability Acceptance Criteria

The traceability matrix is considered complete when:

- all business requirements are mapped
- all functional requirements are linked
- critical business rules are included
- cross-module dependencies are visible
- no requirement exists in isolation

11. System Acceptance Criteria

11.1 Overview

This section defines the conditions under which the LogisticsPro API will be considered complete and acceptable for deployment. Acceptance criteria provide measurable and verifiable conditions that ensure the system satisfies business requirements, functional expectations, data integrity, security controls, and operational reliability.

Acceptance criteria are defined at:

- system level
- module level

- business flow level
- non-functional level

These criteria will be used for validation, testing, stakeholder review, and final approval.

11.2 System-Level Acceptance Criteria

The system shall be considered acceptable when:

- all defined business requirements are implemented and traceable
- all functional requirements behave as specified
- all business rules are enforced without contradiction
- no critical defects exist in core business or financial operations
- data integrity is maintained across all modules
- authentication and authorization controls are enforced
- API responses are consistent and predictable
- system behavior remains stable under normal operating conditions

11.3 Module-Level Acceptance Criteria

11.3.1 Customer and Vendor Management

The module is acceptable when:

- customers and vendors can be created, updated, and retrieved successfully
- invalid or incomplete data is rejected
- duplicate or inconsistent records are prevented according to rules
- inactive entities cannot be used in new operations

11.3.2 Event Management

The module is acceptable when:

- events can be created and retrieved
- invalid date ranges are rejected
- events are correctly linked to job cards where applicable

11.3.3 Job Card Management

The module is acceptable when:

- job cards can be created only with valid customer references
- job status transitions follow defined rules
- closed jobs cannot be modified
- job cost is accurately maintained

11.3.4 Service Management

The module is acceptable when:

- services can be added, updated, and removed for valid jobs
- invalid service data is rejected
- services cannot be added to closed jobs
- service cost contributes correctly to job total

11.3.5 Batch Processing

The module is acceptable when:

- service items can be grouped into batches
- duplicate or invalid items are not allowed in a batch
- batch creation does not corrupt underlying data

11.3.6 Payment Voucher Management

The module is acceptable when:

- vouchers can be created only from valid batches

- payable amounts are calculated correctly
- invalid or duplicate voucher creation is prevented

11.3.7 Invoice Management

The module is acceptable when:

- invoices can be generated from valid jobs
- invoice amounts match underlying service data
- invalid invoice operations are rejected
- void operations preserve history

11.3.8 Receipt Management

The module is acceptable when:

- receipts can be recorded against valid invoices
- overpayment is prevented
- invoice balances are updated correctly

11.3.9 Reporting

The module is acceptable when:

- reports reflect accurate operational and financial data
- filters and selection criteria behave correctly
- no misleading or incomplete data is returned as valid results

11.4 Business Flow Acceptance Criteria

End-to-End Flow: Job to Payment

The system is acceptable when:

1. customer is created or selected
2. job card is created

3. services are added
4. job cost is calculated correctly
5. batch is created (if applicable)
6. invoice is generated
7. payment is recorded via receipt
8. all records remain consistent across modules

Any break in this chain = system not acceptable

11.5 Data Integrity Acceptance Criteria

The system must demonstrate that:

- all entity relationships remain valid
- no orphan records exist
- financial values are consistent across job, invoice, voucher, and receipt
- invalid state transitions are prevented
- data remains consistent after failures or rejected operations

11.6 Security Acceptance Criteria

The system is acceptable when:

- protected endpoints cannot be accessed without authentication
- invalid or expired tokens are rejected
- unauthorized operations are blocked
- sensitive data is not exposed in responses or errors

11.7 API Behavior Acceptance Criteria

The API is acceptable when:

- endpoints follow consistent request/response structure
- status codes correctly reflect outcomes
- validation errors are clearly communicated
- not-found conditions are handled properly
- business rule violations return meaningful responses

11.8 Performance and Stability Acceptance Criteria

The system must demonstrate:

- stable behavior under normal concurrent usage
- acceptable response time for standard operations
- no data corruption under load
- controlled handling of long-running operations

11.9 Exception Handling Acceptance Criteria

The system is acceptable when:

- invalid inputs are rejected without partial processing
- failures do not leave inconsistent data
- system errors do not expose sensitive information
- dependency failures are handled safely

11.10 Regression and Consistency Acceptance Criteria

The system must ensure:

- new changes do not break existing functionality
- existing workflows remain valid after updates
- cross-module consistency is preserved

11.11 Final Acceptance Condition

The LogisticsPro API will be accepted when:

- all sections of this document are satisfied
- all critical business flows execute successfully
- financial operations are accurate and auditable
- no critical or high-severity issues remain unresolved
- stakeholders confirm that system behavior aligns with business expectations

12. Appendix

12.1 Overview

This appendix provides supporting information required to fully understand the LogisticsPro API Business Analysis Document. It includes definitions of key terms, abbreviations, and reference structures used throughout the document. The purpose of this section is to ensure clarity, consistency, and shared understanding among stakeholders, including developers, testers, reviewers, and business users.

12.2 Glossary of Terms

Term	**Definition**
Customer	A business entity that receives services and is billed through the system
Vendor	A service provider such as a transport provider, hotel, or visa service provider
Event	A contextual grouping of operational activities, optionally linked to job cards
Job Card	A unit of operational work associated with a customer, representing a set of services

Service Item	An operational activity (transport, hotel, visa, or miscellaneous) linked to a job
Batch	A grouping of service items prepared for financial processing
Payment Voucher	A record representing payment obligation to a vendor
Invoice	A billing document issued to a customer based on job-related services
Proforma Invoice	A preliminary invoice generated before final billing
Receipt	A record of payment received from a customer
Status	A field representing the current state of an entity (e.g., Open, Closed, Active, Void)
API	Application Programming Interface used for communication between systems
Endpoint	A specific API URL used to perform an operation
Authentication	Process of verifying the identity of a user or system
Authorization	Process of determining access rights to specific operations
Validation	Process of checking data correctness before processing
Business Rule	A constraint or condition that governs system behavior
Transaction	A unit of work that must complete fully or not at all

12.3 Abbreviations

Abbreviation	**Meaning**
API	Application Programming Interface
BA	Business Analysis

BR	Business Requirement
FR	Functional Requirement
BRL	Business Rule
NFR	Non-Functional Requirement
RTM	Requirements Traceability Matrix
ID	Identifier
UI	User Interface

12.4 Entity Reference Summary

This section provides a quick reference of key entities defined in the system.

Entity	**Purpose**
Customer	Stores customer details
Vendor	Stores vendor/service provider details
Event	Represents event grouping
Job Card	Represents operational work unit
Service Item	Represents operational service
Batch	Groups services for financial processing
Payment Voucher	Tracks vendor payments
Invoice	Represents customer billing
Proforma Invoice	Preliminary billing record
Receipt	Tracks payments received

12.5 Status Definitions

Job Status

Status	Description
Open	Job is active and editable
Closed	Job is finalized and cannot be modified

Invoice Status

Status	Description
Draft	Invoice created but not finalized
Issued	Invoice sent/confirmed
Void	Invoice cancelled but retained for history

Invoice Status

Status	Description
Draft	Invoice created but not finalized
Issued	Invoice sent/confirmed
Void	Invoice cancelled but retained for history

General Entity Status

Status	Description
Active	Available for use
Inactive	Not allowed for new operations

General Entity Status

Status	Description
Active	Available for use

Inactive	Not allowed for new operations

12.6 Error Category Definitions

Category	**Description**
Validation Error	Input data is incorrect or incomplete
Business Rule Error	Operation violates defined business rules
Authorization Error	User lacks permission
Authentication Error	User not authenticated
Not Found Error	Requested resource does not exist
System Error	Unexpected internal failure

12.7 Document Structure Summary

This document is organized into the following sections:

1. Introduction
2. Business Requirements
3. Functional Requirements
4. Business Rules
5. Business Process Flows
6. Data Requirements
7. Non-Functional Requirements
8. API Requirements
9. Assumptions, Risks, Dependencies, Exception Handling
10. Traceability Matrix
11. Acceptance Criteria

12.8 Usage Guidelines

- This document must be used as the primary reference for system design and implementation
- All changes to system behavior must be reflected in this document
- Any ambiguity must be resolved before implementation
- Developers, testers, and stakeholders must align with definitions provided here